UNIVERSITY OF GHANA

COLLEGE OF BASIC AND APPLIED SCIENCES

SCHOOL OF ENGINEERING SCIENCES

DEPARTMENT OF COMPUTER ENGINEERING

SECOND SEMESTER 2025/2026 ACADEMIC YEAR

COURSE CODE AND TITLE: CPEN 438 – ADVANCED COMPUTER ORGANIZATION AND ARCHITECTURE

PROJECT TITLE: WIRING THE DATA CENTRE: INTERCONNECTION NETWORKS FOR A SIMULATED ACCRA WAREHOUSE-SCALE CLUSTER

WEEK 3 DESIGN & EXPERIMENTAL SPECIFICATION

GROUP 1

GROUP MEMBERS:

1. **AYERTEY VANESSA ESINAM - 11264010**
2. **PEGGY ESINAM SOMUAH - 11049523**
3. **BERNARDINE ADUSEI-OKRAH – 11123762**
4. **MUHAMMAD NURUL HAQQ ABDUL BASIT MUNAGAH – 11117536**
5. **FAROUK SEDICK - 10947554**

LECTURER: PROF. ROBERT SOWAH

DUE DATE: 23RD AUGUST, 2026

WEEK 3 DESIGN & EXPERIMENTAL SPECIFICATION

FIELD	VALUE
Week 3 project-specific objective	Integrate the verified Week 2 topology and routing module into a cycle-accurate flit-level simulator; implement the seeded uniform-random and hot-node-skewed traffic generators; run the full injection-rate sweep for both topologies under both patterns; measure latency, throughput and the saturation injection rate with explicit packet-loss accounting; and begin the innovation component.
Assigned Group 1 configuration	16 nodes; 16-node ring; 4 × 4 2D mesh; traffic seed 1301; hot database nodes 5 and 1 (derived from the seed); 50% of traffic addressed to them.
Baseline routing	Ring: shortest-direction. Mesh: deterministic X-then-Y dimension-order. Both reused unchanged from Week 2.
Primary Week 3 artefacts	network_sim.c/.h, traffic.c/.h, test_simulator.c, run_sweep.c, gen_datacenter_traffic.py, analyze_network_results.py, bisection_latency_model.m.
Verified Week 3 result	216 runs, 0 conservation violations, 0 packets lost below saturation. Uniform traffic: ring saturates at 0.28 and mesh at 0.70 flits/node/cycle. Hot-node traffic: both fall to 0.20. Innovation reduces background latency 71× (mesh) and 60× (ring) at the knee.
Repository	github.com/somuahes/CPEN438-Project13-Group1
Date prepared	23rd August 2026

1. PURPOSE AND SCOPE

This specification defines the design, implementation approach, modelling assumptions, experimental method and measured results for Group 1's Week 3 deliverable for Project 13. Week 2 established that the topology and routing module is structurally correct. Week 3 puts that module under load: it adds the cycle-accurate flit-level simulation core, the two seeded warehouse-scale traffic generators, and the measurement harness that turns structural properties such as diameter and bisection bandwidth into quantitative statements about latency, throughput and saturation.

The scope of Week 3 covers the flit and packet model, the virtual-channel router and its flow control, the integration of the Week 2 routing functions into the per-cycle simulation loop, the uniform-random and hot-node-skewed traffic generators driven by Group 1's seed 1301, the injection-rate sweep across both topologies and both traffic patterns, the measurement of average packet latency and accepted throughput, the identification of each configuration's saturation injection rate, explicit accounting for every generated packet, an analytical channel-load model validated against the measurements, and the first implementation and evaluation of the creativity and innovation component.

The folded-torus extension and the adaptive-routing research challenge are outside the scope of this specification and are reserved for Week 4, as is the final project report and the defence rehearsal. No claim is made in this document about topologies or routing algorithms that have not been implemented and measured.

1.1 Week 3 deliverables checklist

REQUIREMENT	EVIDENCE IN THIS SUBMISSION	STATUS
Flit-level simulator integrated with the Week 2 topology module	student_implementation/network_sim.c and .h	Complete
Seeded uniform-random traffic generator	traffic.c/.h; python/gen_datacenter_traffic.py	Complete
Seeded hot-node-skewed traffic generator	traffic.c/.h; hot set {5, 1} derived from seed 1301	Complete
Full injection-rate sweep, both topologies, both patterns	tests/run_sweep.c; 216 runs; week3_sweep_results.csv	Complete
Average latency and achieved throughput measurement	week3_sweep_results.csv; Figures 1–3	Complete
Saturation injection rate per configuration	week3_saturation_summary.csv; Figure 7; Table in §11	Complete
Saturation-curve plots	Figures 1, 2 and 4	Complete
Explicit packet-loss accounting	Conservation identity checked at all 216 points; Figure 6	Complete
Analytical model validated against measurement	matlab/bisection_latency_model.m; Figure 4; §12	Complete
Unit tests	tests/test_simulator.c — 97 checks across 12 test functions	Complete
Innovation component begun	End-to-end traffic-class separation; Figure 5; §13	Complete
Commit history	Repository commit log (§15.2)	Complete
Folded torus / adaptive routing	Requires Week 4 implementation	Deferred to Week 4

2. REQUIREMENTS TRACEABILITY

The table below connects the Project 13 requirements to the concrete Week 3 artefacts, and states plainly which requirements remain deferred.

PROJECT REQUIREMENT	WEEK 3 DESIGN RESPONSE	EVIDENCE / STATE
Flit-level packet model with configurable injection rate and per-hop router delay	Packets are four flits (head, two body, tail); one flit per link per cycle per direction; one cycle of router delay per hop; injection is a per-node Bernoulli process at rate λ/4 packets per cycle.	Implemented + tested
Correct packet injection, routing and delivery for at least two topologies	The simulator obtains every route from the unchanged Week 2 topology_route(), memoised into a next-hop table at start-up.	Implemented + tested
Uniform-random traffic generator	traffic_dest() draws uniformly over all nodes except the source, with no rejection step.	Implemented + tested
Hot-node-skewed traffic generator	A seed-derived pair of database nodes receives 50% of all packets; the remainder is uniform.	Implemented + tested
Average latency and achieved throughput across a sweep of injection rates	18 injection rates from 0.02 to 0.80 flits/node/cycle, for 12 configurations.	Measured

PROJECT REQUIREMENT	WEEK 3 DESIGN RESPONSE	EVIDENCE / STATE
Saturation injection rate identified per topology	A point is saturated when accepted throughput falls below 95% of offered load, or when any packet is dropped.	Measured
Zero packet loss below saturation, with loss explicitly tracked	The conservation identity is asserted at every sweep point; the only loss path is a full source queue and it is counted.	Verified
Latency-vs-injection-rate curves showing the saturation point	Figures 1, 2 and 4.	Produced
Demonstration that hot-node traffic exposes a bisection-bandwidth limit	§9 and §12: the binding constraint moves from the bisection cut to the channels feeding the database nodes.	Demonstrated
Analytical MATLAB model validated against measured data	bisection_latency_model.m; validation table in §12.	Validated
Reproducibility of every run	configs/group1_week3_config.txt records node count, topology, routing, seed, model parameters and sweep parameters.	Recorded
Innovation: virtual channels evaluated on the hot-node pattern	End-to-end traffic-class separation, with two controls (§13).	Implemented + measured
Folded torus; adaptive routing	Belongs to the Level-3 extension phase.	Deferred to Week 4

2.1 Assigned configuration and Week 3 model parameters

PARAMETER	GROUP 1 VALUE	DESIGN SIGNIFICANCE
Node count	16	The same node count is used for both topologies, so the comparison is fair.
Ring / mesh	16 nodes / 4 × 4	Diameter 8 and 6; bisection bandwidth 2 and 4 links (verified in Week 2).
Traffic seed	1301	Fixes both the destination stream and the hot-node set.
Hot database nodes	5 and 1	Derived as seed mod N and (seed div N) mod N, so the set is team-specific rather than hard-coded.
Hot traffic fraction	0.50	Half of all packets are addressed to 12.5% of the nodes.
Packet size	4 flits	One head, two body, one tail; long enough for wormhole behaviour to matter.
Link bandwidth	1 flit/cycle/direction	Unit channel width, so channel load can be read directly as a saturation bound.
Router delay	1 cycle per hop	Makes zero-load latency directly comparable with the Week 2 hop counts.
VC buffer depth	4 flits	Equal to the packet length, so one packet fits in one virtual channel.
Baseline VCs per port	2	The minimum the ring needs for deadlock freedom; the mesh is given the same budget.
Ejection bandwidth	4 flits/cycle	Set to the maximum node degree so that ejection is not the bottleneck under hot-node traffic (§4.4).

PARAMETER	GROUP 1 VALUE	DESIGN SIGNIFICANCE
Source-queue capacity	1024 packets per node	Held constant across all VC modes, so no mode wins by being given more memory.
Warmup / measurement	3000 / 12000 cycles	Warmup discards the fill transient; measurement is long enough for stable averages.
Injection rates swept	0.02 … 0.80 flits/node/cycle	18 points, finely spaced through the saturation knee.

3. SIMULATOR ARCHITECTURE AND INTEGRATION BOUNDARY

Week 2 defined a clean boundary: the topology module determines where a packet should move, and the simulation core determines when it moves and how network load affects its progress. Week 3 implements the second half of that boundary without disturbing the first. The topology module is included unmodified; the simulator calls topology_route() for every ordered node pair at start-up and stores the resulting first hop in a next-hop table. Route lookups during simulation are then a single array access, but the routes themselves are, by construction, exactly the routes the Week 2 verifier printed. If a routing bug were introduced into the Week 2 module, the Week 2 unit tests, the Week 2 manual traces and the Week 3 traffic results would all move together rather than silently diverging.

The Week 3 module therefore owns four things: the flit and packet representation, the virtual-channel router and its flow control, the per-cycle simulation loop, and the measurement and accounting instrumentation. Traffic generation lives in a separate module so that the same simulator can be driven by either pattern, or later by a trace file, without any change to the core. The analysis and plotting stage is entirely outside the C code and consumes only the CSV files the sweep driver writes.

4. FLIT-LEVEL NETWORK MODEL

4.1 Packet and flit model

A packet is four flits: one head flit, two body flits and one tail flit. The head flit carries the destination and performs route computation and virtual-channel allocation; the body and tail flits inherit that allocation and follow the head in order, which is what makes the model wormhole rather than store-and-forward. The tail flit releases the virtual channel as it leaves. Every link carries one flit per cycle in each direction, so a channel's capacity is exactly one flit per cycle and a channel load computed in units of the injection rate can be inverted directly into a saturation bound.

Packets are generated at each node by an independent Bernoulli trial each cycle with probability $\lambda/4$, where λ is the offered load in flits per node per cycle. A generated packet joins that node's source queue and is stamped with its generation cycle. End-to-end latency is measured from that stamp to the cycle in which the tail flit is ejected at the destination, so it includes source queueing delay, injection serialisation and all network contention. Measuring latency from injection into the network rather than from generation would hide precisely the queueing growth that defines saturation.

4.2 Router microarchitecture and the per-cycle pipeline

Each node holds one input-buffered virtual-channel router. There is one input port per topology neighbour, at most four, plus one local injection port; ejection is a separate output port. Each input port carries V virtual channels of four flit slots each. Flow control is credit-based: a flit advances only when the downstream virtual channel has a free slot at the start of the cycle.

Each cycle is evaluated in two phases so that no flit can traverse two routers within a single cycle, which is the most common way a cycle-accurate simulator quietly overstates throughput.

Credits and grants are computed entirely from the start-of-cycle state, and only then are all granted movements committed:

```
1. Snapshot the free space in every virtual channel.
2. Route computation and VC allocation for newly arrived head flits.
3. Switch allocation: one flit per output port per cycle, chosen by
   round-robin arbitration among the requesting input VCs. Ejection
   is granted up to EJECT_BW flits per node per cycle.
4. Commit every granted movement, releasing the source VC when a
   tail flit departs.
5. Generate new packets and serialise queued packets into the local
   input port, one flit per VC per cycle.
```

4.3 Deadlock freedom and the dateline rule

The 2D mesh under strict XY dimension-order routing has an acyclic channel-dependency graph and is deadlock-free with a single virtual channel. A bidirectional ring under shortest-direction routing is not: the clockwise channels alone form a cycle, and although no individual packet travels more than eight hops, a chain of blocked packets can close that cycle. This was observed during development as a hard hang at high offered load.

The fix is the standard dateline rule from Dally and Towles. The ring is given two virtual channels per port; a packet starts on the low channel and is forced onto the high channel the moment it crosses the link joining node 15 and node 0. Because shortest-direction routing crosses that link at most once, the dependency cycle is broken. The important design consequence is that the mesh is then given the same two-channel budget even though it does not need it, so that the ring-versus-mesh comparison measures topology rather than buffer allocation. A unit test asserts that the ring still makes forward progress at an offered load of 0.80, which would fail immediately if the rule were removed.

One consequence of the rule should be stated plainly rather than glossed over. Equalising the virtual-channel budget equalises the count, not the usable concurrency: because the ring's two channels are partitioned by the dateline, a ring packet may only ever occupy one of them at a time, whereas a mesh packet may use either. The ring therefore has two channels but one usable channel per packet, and the mesh has two of each. This asymmetry is inherent to making a ring deadlock-free at all and cannot be removed by allocating buffers differently. It is reported alongside the results so that the ring-versus-mesh comparison is read for what it is: a comparison of two topologies each given the minimum flow control it needs to operate correctly, rather than a perfectly matched control. The measured gap is in any case dominated by bisection bandwidth and hop count — the mesh carries a peak channel load of 1.07 against the ring's 2.40 under uniform traffic — rather than by channel concurrency.

4.4 Modelling assumptions and their justification

ASSUMPTION	JUSTIFICATION AND CONSEQUENCE
Ejection bandwidth is four flits per node per cycle rather than one.	With single-flit ejection, a hot node addressed by half of all traffic saturates on its own ejection port at the same rate in both topologies, and the experiment would measure the processing element rather than the network. Setting ejection to the maximum node degree makes the binding constraint the links entering the hot node — two on the ring, three or four on the mesh

ASSUMPTION	JUSTIFICATION AND CONSEQUENCE
	— which is exactly the topology property the project asks us to expose. This is stated explicitly because it materially affects the hot-node results.
Source queues are bounded at 1024 packets and overflow is counted as loss.	An unbounded queue would never lose a packet, which would make the required loss accounting vacuous. A bounded queue gives a well-defined, countable loss event that is provably zero below saturation.
Credit information is one cycle old.	This is how real credit-based flow control behaves and it prevents same-cycle multi-hop traversal. It slightly understates achievable throughput, which is the conservative direction.
Router delay is one cycle per hop, with no separate pipeline stages.	Keeps zero-load latency directly comparable with the Week 2 hop counts. Adding pipeline depth would shift all curves right by a constant and would not change any comparison.
Bisection bandwidth is taken from the Week 2 closed forms.	General minimum bisection is NP-hard. For regular ring and mesh topologies the closed forms are exact, and §12 supplements them with an explicit per-channel load computation that does not assume a symmetric cut.

4.5 Public interface

FUNCTION	CONTRACT / RESPONSIBILITY
sim_create(topology, traffic, seed, hot_fraction, num_hot, mode)	Build a simulator over an already-constructed topology, which is borrowed rather than owned. Builds the reverse-port, next-hop and dateline tables.
sim_step(s)	Advance exactly one cycle through the five-stage pipeline of §4.2.
sim_run(s, rate, warmup, measure, drain_max)	Run warmup, then the measurement window, then a bounded drain in which injection stops and measurement packets retire.
sim_check_conservation(s)	Return true if generated = delivered + dropped + in flight + queued.
sim_accepted_throughput(s)	Flits ejected during the measurement window, per node per cycle.
sim_avg_latency(s), sim_avg_hops(s)	Mean end-to-end latency and hop count over retired measurement packets.
sim_avg_latency_class(s, cls), sim_throughput_class(s, cls)	The same metrics split by traffic class: 0 is background traffic, 1 is traffic addressed to a database node. These are the metrics that expose head-of-line blocking.
traffic_init / traffic_dest / traffic_is_hot	Seeded destination generation and hot-set membership.

4.6 Complexity and resource characteristics

OPERATION	TIME COMPLEXITY	WORKING MEMORY
Next-hop table construction (start-up)	$O(V^2 \times \text{path length})$	$O(V^2)$ integers
One simulation cycle	$O(V \times \text{ports} \times \text{VCs})$	$O(V \times \text{ports} \times \text{VCs} \times \text{buffer depth})$ flits
Route lookup during simulation	O(1)	None
One sweep point (15 000 cycles + drain)	$O(\text{cycles} \times V \times \text{ports} \times \text{VCs})$	Constant in the number of packets
Full 216-run sweep	about 20 seconds at -O2	Under 5 MB

5. TRAFFIC GENERATION AND REPRODUCIBILITY

Both traffic patterns are driven by an xorshift64* generator seeded through SplitMix64. An explicit generator is used in preference to the C library rand() for two reasons: results are then identical on every machine and every C library, and the Python reference generator can reproduce the same stream exactly. The first implementation used a 32-bit linear congruential generator, which biased the Bernoulli injection test at the small probabilities used here and made accepted throughput read about six percent above the offered rate at light load; replacing it removed the bias entirely.

The uniform pattern draws an index in [0, N−1) and shifts it past the source, which is exactly uniform over the fifteen legal destinations and never rejects a draw. The hot-node pattern first tests whether the packet belongs to the hot half of the traffic; if it does, it picks one of the database nodes, and a database node that draws itself falls through to the uniform component rather than being discarded, so that the offered load per node stays exactly at the requested rate.

5.1 Seed-derived hot-node set

The hot database nodes are computed from the seed rather than written into the source, so that a new seed produces a genuinely different workload and the team can generate a fresh pattern on demand at the defence. The rule is hot[0] = seed mod N and hot[1] = (seed div N) mod N, with deterministic forward-walking on a collision. For Group 1's seed 1301 and sixteen nodes this gives 1301 mod 16 = 5 and 81 mod 16 = 1, so the database nodes are node 5, an interior mesh node at (1,1), and node 1, an edge mesh node at (0,1). That the two hot nodes have different mesh degrees is convenient rather than accidental: it is the reason the mesh's hot-node bound in §12 is set by node 1 rather than node 5.

5.2 Cross-language verification

The C generator and the Python reference in gen_datacenter_traffic.py implement the same three shifts, the same multiply and the same order of draws. The support tool tests/dump_traffic.c writes the first three thousand source–destination pairs for each pattern, and the Python script re-derives them:

```
$ ./dump_traffic 3000
hot nodes derived from seed 1301: 5 1
wrote 3000 pairs per pattern to results/traffic_reference_c.csv

$ python3 python/gen_datacenter_traffic.py \
      --verify results/traffic_reference_c.csv
 uniform: 3000/3000 destinations match the C generator
 hotnode: 3000/3000 destinations match the C generator
VERIFIED: the Python and C traffic generators are bit-for-bit identical.
```

The uniform generator was additionally checked for flatness, because the Project 13 instructor notes warn that a generator which quietly concentrates load will defeat the mesh's bisection advantage and make the routing implementation look like the culprit. Over two hundred thousand packets, every node receives within eight percent of one-sixteenth of the traffic, and no packet ever addresses its own source.

6. EXPERIMENTAL DESIGN

6.1 Variables

ROLE	VARIABLE	LEVELS
Independent	Topology	16-node ring; 4 × 4 mesh
Independent	Traffic pattern	uniform-random; hot-node-skewed
Independent	Injection rate	18 levels, 0.02 to 0.80 flits/node/cycle
Independent	Flow-control configuration	baseline (2 VCs, shared queue); control A (4 VCs, shared queue); control B (2 VCs, class queues); innovation (4 VCs, class queues)
Dependent	Average packet latency	cycles, overall and split by traffic class
Dependent	Accepted throughput	flits/node/cycle, overall and split by traffic class
Dependent	Saturation injection rate	flits/node/cycle
Dependent	Packets dropped	count, from full source queues only
Controlled	Seed, packet size, buffer depth, total queue capacity, warmup and measurement length	held constant across every run

6.2 Measurement protocol

Each run consists of 3000 warmup cycles, 12000 measurement cycles, and a drain of up to 30000 cycles in which injection stops and the measurement packets retire. Accepted throughput is counted as flits ejected during the measurement window regardless of when they were generated, which is the standard definition and makes the metric independent of how long the drain is allowed to run. Latency is averaged over packets generated during the measurement window and retired by the end of the drain, and the number that fail to retire is reported alongside. Above saturation this makes the reported average latency a lower bound, which is stated rather than hidden.

6.3 Saturation criterion

A swept point is classified as saturated when accepted throughput falls below 95% of the offered rate, or when any packet is dropped from a source queue. The reported saturation injection rate for a configuration is the highest swept rate that is still stable and loss-free, and the first saturated rate is reported next to it so that the width of the bracket is visible. A throughput-based criterion is used rather than a latency threshold because a latency threshold requires an arbitrary multiplier, whereas the divergence between offered and accepted load is unambiguous.

7. PACKET ACCOUNTING

Project 13 identifies a specific common failure: measuring latency only for successfully delivered packets while silently dropping packets under high load. The simulator is built so that this cannot happen. There is exactly one place in the entire design where a packet can be lost — a full source queue — and that event increments a counter. At every moment the following identity holds:

```
generated == delivered + dropped_full + in_network + queued
```

sim_check_conservation() evaluates this identity by taking a fresh census of every source queue rather than trusting a running total, so a bookkeeping error in the simulator would show up as a mismatch rather than being absorbed. The sweep driver evaluates it after every one of the 216 runs and reports the count of violations. The result was zero violations, and zero packets dropped at any rate at or below the reported saturation point for every configuration. Figure 6 plots the drop count against offered load and shows it pinned at zero until the saturation marker for each curve.

8. RESULTS: UNIFORM-RANDOM TRAFFIC

Under uniform-random traffic the two topologies behave exactly as their static metrics predict, and the gap is large. The mesh is faster when the network is empty because its average hop count is lower — 2.67 hops against the ring's 4.27, giving zero-load latencies of 6.88 and 8.57 cycles — and it stays flat far longer because it has twice the bisection bandwidth and 50% more channels. The ring's latency has already begun to climb by an offered load of 0.20 and turns over sharply just past 0.24, while the mesh is still within a cycle of its unloaded latency at 0.28 and does not turn over until beyond 0.70.

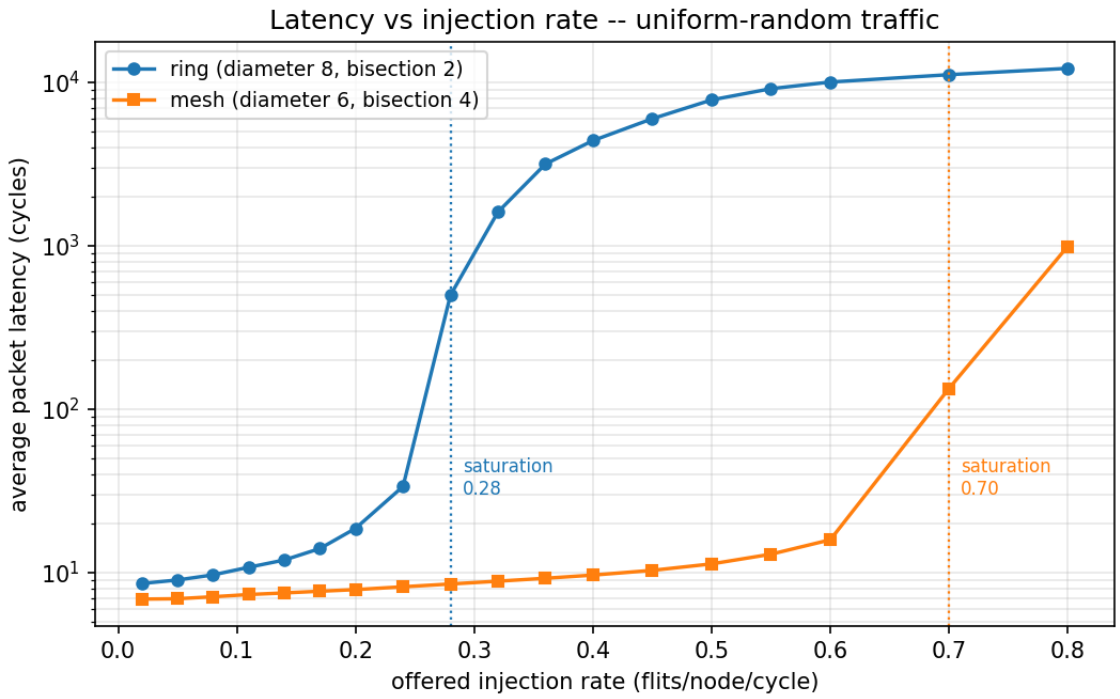

Fig. 1 — Average packet latency against offered injection rate under uniform-random traffic. The dotted markers show the highest loss-free rate for each topology.

The measured saturation rates are 0.28 flits/node/cycle for the ring and 0.70 for the mesh, a factor of 2.5. The peak accepted throughput follows the same ratio: 0.272 against 0.724 flits/node/cycle. This is the Project 13 worked example reproduced quantitatively — the ring's bisection bandwidth of two links becomes the binding constraint well before the mesh's four, so the mesh sustains meaningfully higher injection rates before saturating.

9. RESULTS: HOT-NODE-SKEWED TRAFFIC

The hot-node pattern changes the picture in a way that is more interesting than a simple scaling. Both topologies fall back to a saturation rate of 0.20 flits/node/cycle. The ring loses about 29% of its uniform-traffic capacity; the mesh loses 71%. In other words, the skewed workload does not merely hurt the bisection-limited topology — it erases most of the advantage the mesh had earned under uniform traffic.

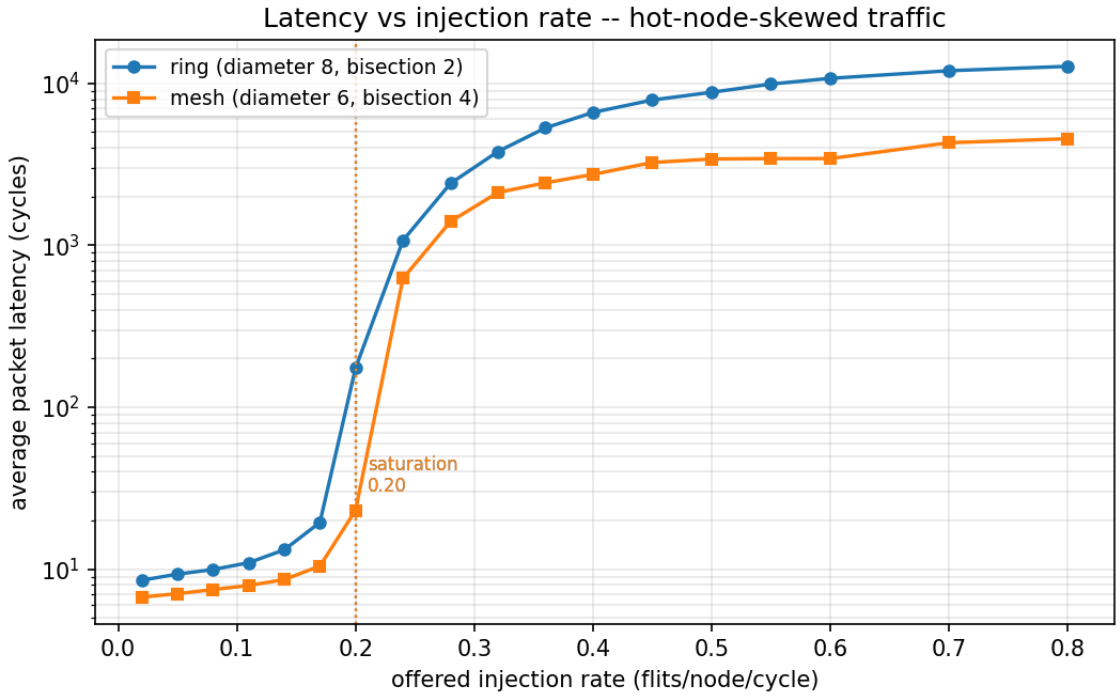

Fig. 2 — Average packet latency under hot-node-skewed traffic, with 50% of all packets addressed to database nodes 5 and 1.

The reason is that the binding constraint has moved. Under uniform traffic the busiest channel in the mesh carries 1.07 units of load per unit of injection rate, and the network is limited by its overall cross-sectional bandwidth. Under hot-node traffic the busiest mesh channel carries 4.53 units, because dimension-order routing funnels everything addressed to node 1 and node 5 down the same column of links. The classical bisection cut is no longer where the network breaks; the few channels entering the database nodes are. The ring is affected less in relative terms only because it had so much less to lose: its busiest channel goes from 2.40 to 4.20 units.

This is the demonstration Project 13 asks for, and it carries a design lesson worth stating at the defence. Bisection bandwidth is the right first-order metric for uniform traffic, but for realistic

skewed data-centre workloads the relevant quantity is the local bandwidth available at the hot destinations. A topology chosen purely on bisection bandwidth can be disappointing in production for exactly this reason.

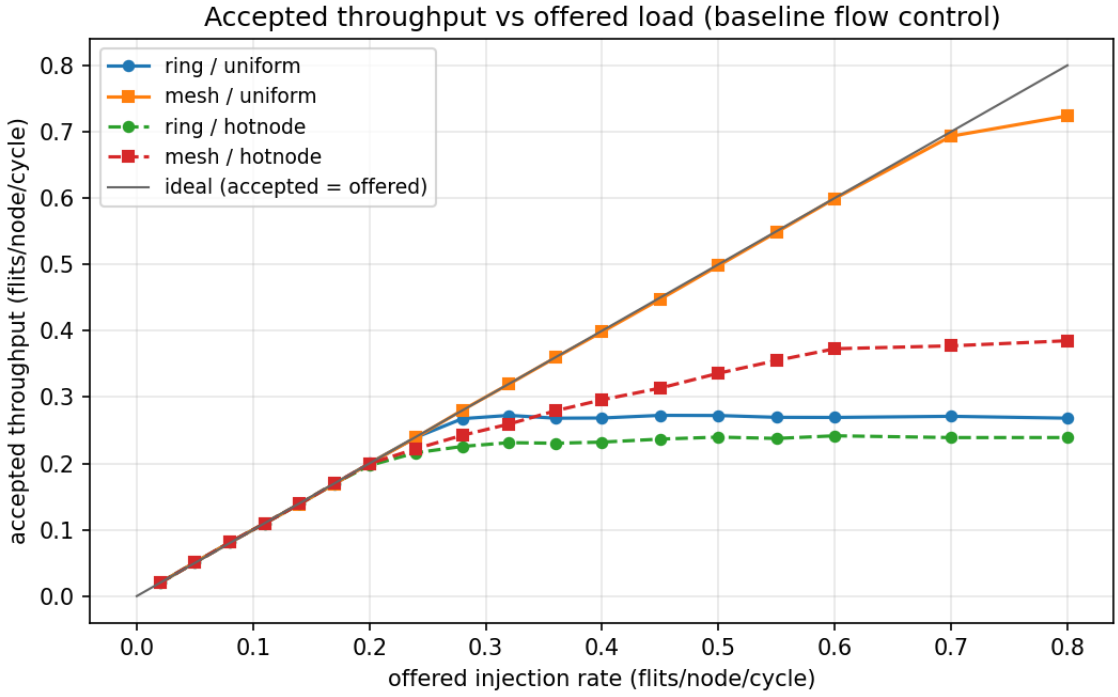

Fig. 3 — Accepted throughput against offered load. Each curve tracks the ideal diagonal until its network can no longer absorb the offered rate, then flattens.

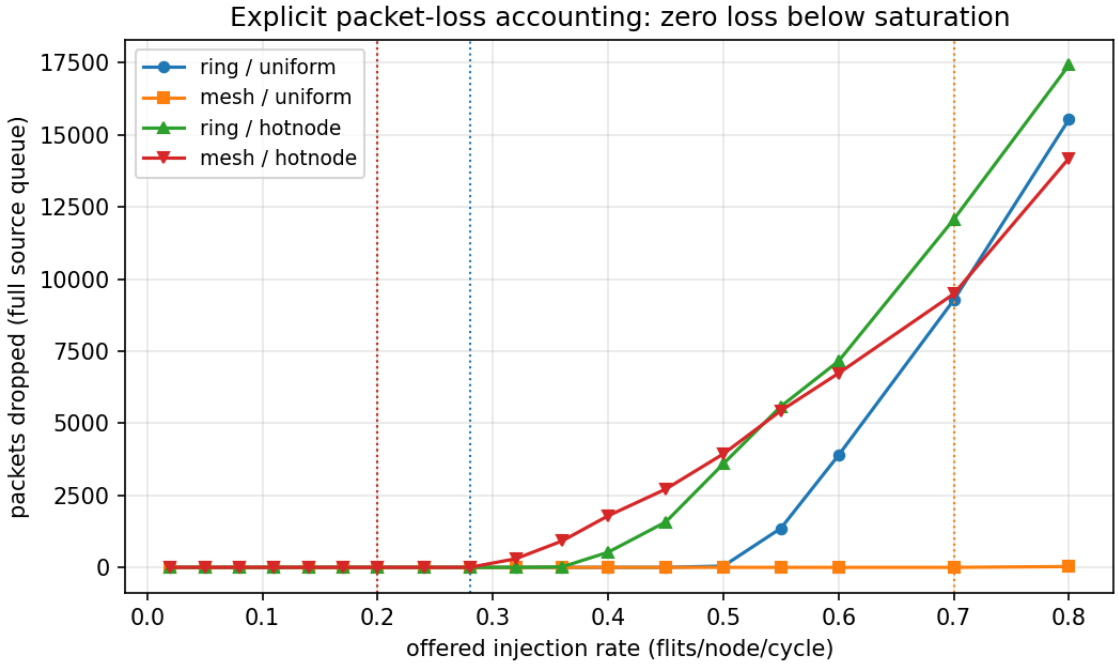

Fig. 6 — Explicit packet-loss accounting. Losses are exactly zero until past each configuration's saturation marker; every lost packet is a counted source-queue overflow.

10. REQUIRED COMPARISON TABLE

The table required by Project 13 Section L relates each topology's static metrics to its measured saturation behaviour.

TOPO-LOGY	TRAFFIC	DIA-METER	BISECT. (links)	AVG HOPS	ZERO-LOAD LATENCY (cycles)	SATUR-ATION RATE	PEAK THROUGH-PUT	LATENCY AT λ = 0.20
Ring(16)	uniform	8	2	4.27	8.57	0.28	0.272	18.6
Mesh(4×4)	uniform	6	4	2.67	6.88	0.70	0.724	7.9
Ring(16)	hot-node	8	2	4.27	8.56	0.20	0.242	174.7
Mesh(4×4)	hot-node	6	4	2.53	6.73	0.20	0.385	23.1

Three readings are worth drawing out. First, zero-load latency is set almost entirely by diameter, through the average hop count — the mesh is 1.7 cycles faster before any contention exists. Second, saturation under uniform traffic is set by bisection bandwidth, and the 2.5× ratio in saturation rate is close to the 2× ratio in bisection bandwidth combined with the 1.6× ratio in average hop count. Third, at an offered load of 0.20 the mesh is still delivering packets in 7.9 cycles under uniform traffic but takes 23.1 under hot-node traffic, while the ring degrades from 18.6 to 174.7. The same offered load produces a nine-fold latency penalty on the ring and a three-fold penalty on the mesh, which is the traffic-pattern sensitivity the rubric asks to see demonstrated.

11. ANALYTICAL MODEL AND VALIDATION

The MATLAB model in bisection_latency_model.m predicts behaviour from topology structure alone, so that the simulator can be checked against something independent of itself. For each topology and traffic pattern it walks every source–destination pair through the actual routing algorithm, weights each path by the actual destination probability, and accumulates the load carried by every unidirectional channel. The ideal saturation rate is the reciprocal of the busiest channel's load, capped by the ejection bandwidth of the busiest destination. Latency is then modelled as a zero-load term plus an M/D/1 source-queue term:

```
T0    = H x router_delay + (L - 1)
T(l)  = T0 + (rho / (1 - rho)) x L/2,     rho = l / l*
l*    = min( 1 / gamma , EJECT_BW / max_eject_load )
```

The classical bisection bound is computed alongside for comparison, because it is the bound the Project 13 worked example uses. The comparison is instructive: for uniform traffic the two agree closely, but for hot-node traffic the classical bound is wildly optimistic — it predicts 0.469 for the ring and 0.938 for the mesh, against measured values of 0.20 for both. Only the explicit channel-load walk finds the real constraint.

TOPO-LOGY	TRAFFIC	PEAK CHANNEL LOAD	CHANNEL BOUND	EJECTION BOUND	CLASSICAL BISECTION BOUND	MODEL λ*	MEASURED λ	EFFIC-IENCY
Ring(16)	uniform	2.400	0.417	4.000	0.469	0.417	0.28	0.67
Mesh(4×4)	uniform	1.067	0.938	4.000	0.938	0.938	0.70	0.75
Ring(16)	hot-node	4.200	0.238	0.938	0.469	0.238	0.20	0.84

TOPO-LOGY	TRAFFIC	PEAK CHANNEL LOAD	CHANNEL BOUND	EJECTION BOUND	CLASSICAL BISECTION BOUND	MODEL λ*	MEASURED λ	EFFIC-IENCY
Mesh(4×4)	hot-node	4.533	0.221	0.938	0.938	0.221	0.20	0.91

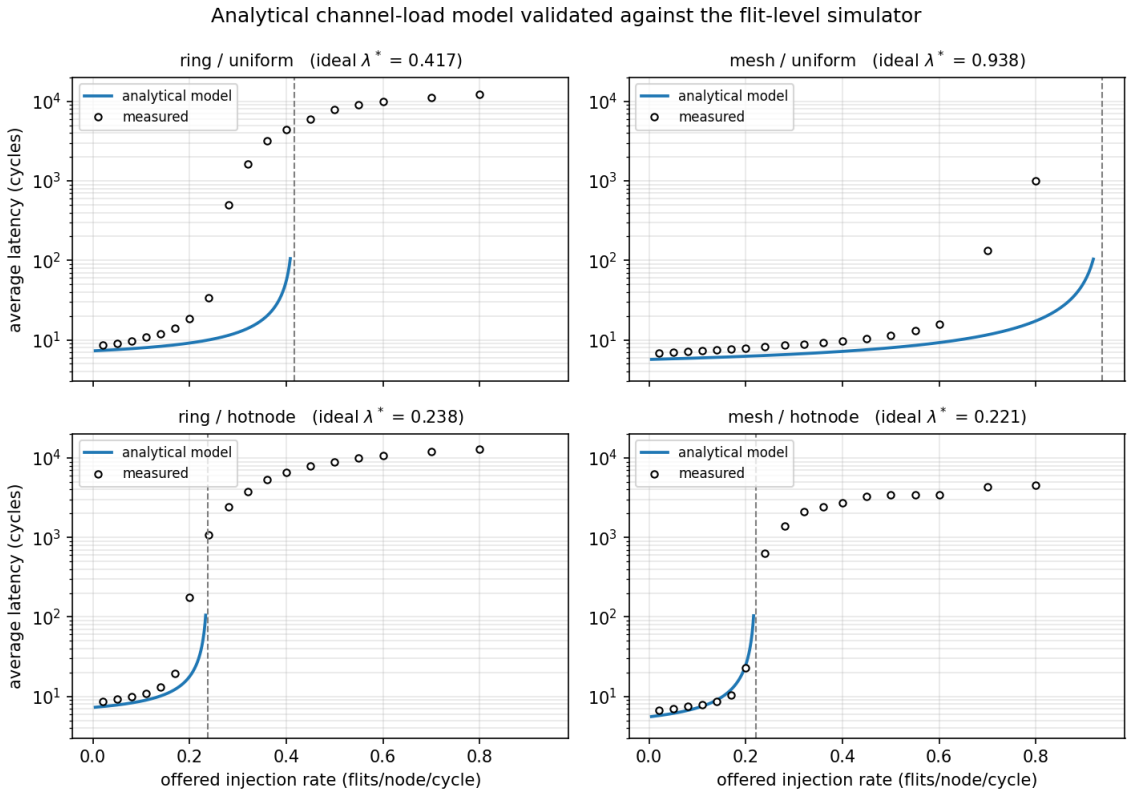

Fig. 4 — The analytical channel-load model against the flit-level measurements. The dashed line marks the model's ideal saturation rate.

The model tracks the hot-node cases closely, reaching 84% and 91% of the ideal bound, because in those cases a single channel is so heavily loaded that it dominates everything else and the network is genuinely limited by one link. It over-predicts the uniform cases, where the measured network reaches only 67% and 75% of the ideal. That gap is not a defect in either the model or the simulator: the model assumes perfect channel utilisation, whereas the real network loses throughput to finite virtual-channel counts, wormhole head-of-line blocking and arbitration conflicts. An achieved efficiency of two-thirds to three-quarters of the ideal channel-load bound is the normal range reported for wormhole networks with shallow buffers, and the fact that efficiency rises towards 0.9 exactly when a single channel becomes the bottleneck is itself evidence that the simulator is behaving correctly.

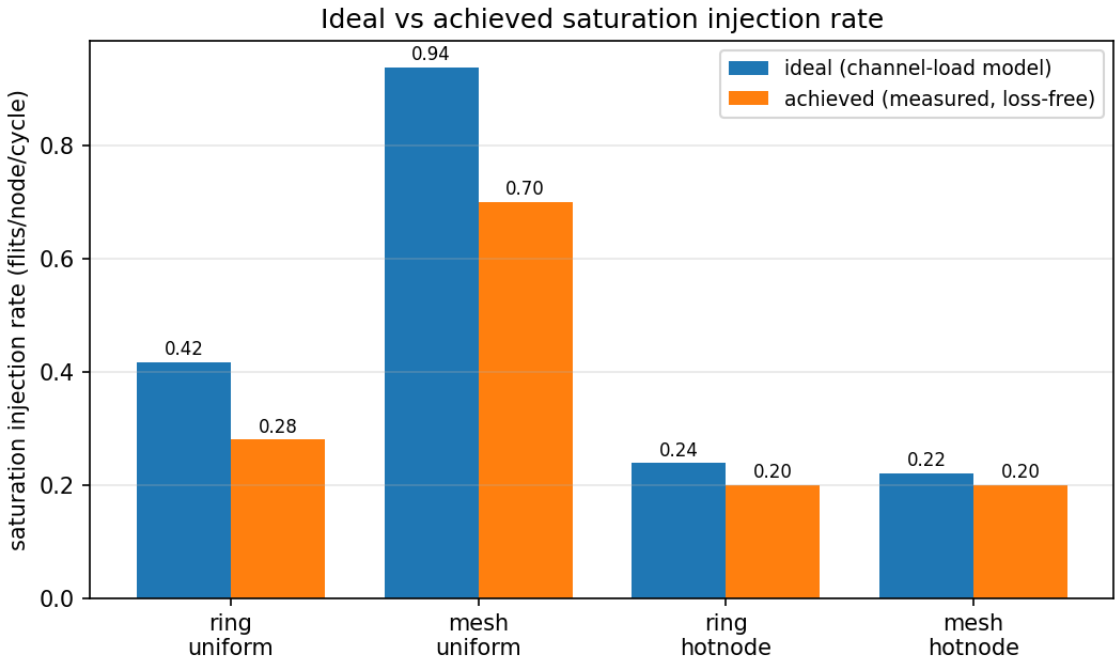

Fig. 7 — Ideal saturation rate from the channel-load model against the measured loss-free rate.

12. INNOVATION: END-TO-END TRAFFIC-CLASS SEPARATION

12.1 The problem the scheme addresses

Under hot-node traffic the packets addressed to the database nodes back up, and that backpressure propagates into buffers that are shared with everything else. A background packet whose own route is completely free is stalled simply because it is sitting behind a hot-destined packet. This is head-of-line blocking, and Project 13's innovation brief names it directly as the phenomenon a virtual-channel scheme should remove. The measured baseline shows how severe it is: at an offered load of 0.24 on the mesh, background packets that need only two or three free hops are taking 574 cycles.

12.2 The scheme

The scheme partitions every shared buffering resource in the machine into two classes, hot-destined and background, at both points where head-of-line blocking can occur. The router input buffers are split so that a hot packet can never occupy a virtual channel a background packet needs, and each node's source queue is split into one queue per class so that a backed-up hot flow cannot block background injection either. Each class receives exactly the baseline's per-port virtual-channel budget, and the total source-queue capacity is held constant, so the scheme is not simply being handed more memory.

An earlier version partitioned only the router virtual channels and showed almost no benefit, because the shared source queue still blocked background packets before they entered the network. That failure is the reason the final scheme is end-to-end, and it is also the reason two controls were added rather than one.

12.3 Experimental design and controls

A scheme that adds virtual channels and then reports an improvement has not shown that isolation is what helped; it may simply have added buffers. Two controls separate the two effects, and both were swept over the same eighteen injection rates as the baseline and the innovation.

CONFIGURATION	VCs PER PORT	SOURCE QUEUES	WHAT IT ISOLATES
Baseline	2	1 shared	The reference point.
Control A	4	1 shared	The effect of doubling buffering with no isolation at all.
Control B	2	2 per class	The effect of the injection-side split alone.
Innovation	4	2 per class	Isolation at both points, with each class holding only the baseline's budget.

12.4 Results

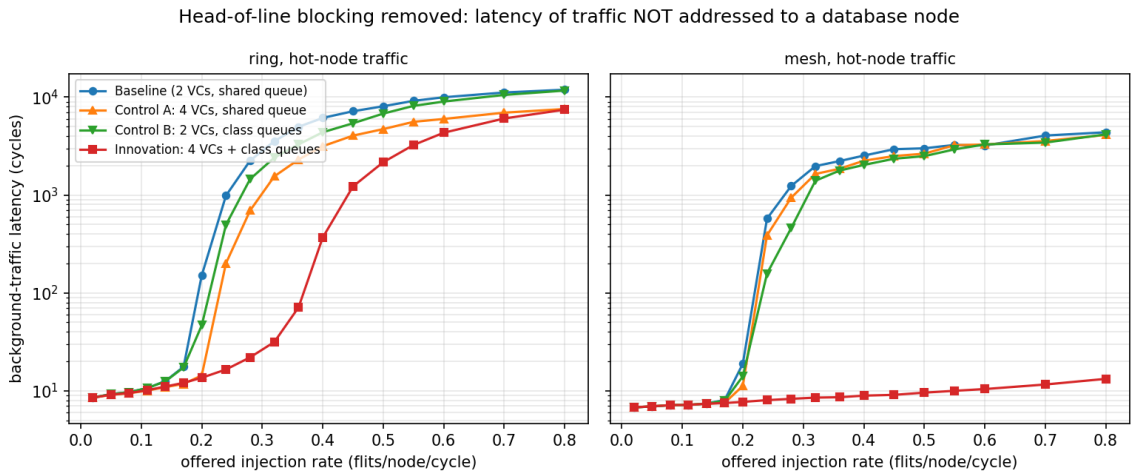

Fig. 5 — Latency of background traffic, meaning packets not addressed to a database node, under hot-node-skewed traffic.

The table below reports every configuration at an offered load of 0.24 flits/node/cycle, which is the first rate past the hot-node saturation knee and therefore where head-of-line blocking is most damaging.

TOPOLOGY	CONFIGURATION	BACKGROUND LATENCY (cycles)	HOT-FLOW LATENCY (cycles)	PEAK ACCEPTED THROUGHPUT	BACKGROUND SPEED-UP
Mesh(4×4)	Baseline (2 VC, shared queue)	574.0	670.5	0.385	—
Mesh(4×4)	Control A (4 VC, shared queue)	387.3	461.0	0.397	1.5×
Mesh(4×4)	Control B (2 VC, class queues)	157.4	646.5	0.395	3.6×
Mesh(4×4)	Innovation (4 VC, class queues)	8.0	626.3	0.568	71.4×
Ring(16)	Baseline (2 VC, shared queue)	984.7	1152.1	0.242	—
Ring(16)	Control A (4 VC, shared queue)	200.3	244.2	0.312	4.9×
Ring(16)	Control B (2 VC, class queues)	495.7	1335.2	0.248	2.0×
Ring(16)	Innovation (4 VC, class queues)	16.5	856.3	0.302	59.6×

Neither control comes close to the full scheme. On the mesh, doubling the buffers alone recovers a factor of 1.5 and splitting the queues alone recovers 3.6, but the two together recover 71.4. The effect is strongly super-additive because head-of-line blocking has to be removed at both points to be removed at all: relieving the routers while the injection queue still blocks, or relieving the injection queue while the routers still block, leaves the chain intact. That is the substantive claim of the innovation, and it is the reason the controls are part of the deliverable rather than an afterthought.

Two honest qualifications belong with the result. First, the hot flow itself is not made faster, and no such claim is made: it is limited by the physical bandwidth of the links entering the database nodes, and no buffering scheme can change that. What the scheme does is stop that unavoidable congestion from spreading to traffic that has nothing to do with it. Second, on the ring the innovation does not raise the loss-free saturation rate, because splitting the source

queue halves each class's depth and one packet overflows at 0.24; the gain there shows up in background latency and in peak accepted throughput, which rises from 0.242 to 0.302. On the mesh the loss-free rate does improve, from 0.20 to 0.24, and peak accepted throughput rises 48%, from 0.385 to 0.568.

As a sanity check, the scheme was also swept under uniform-random traffic, where there is only one traffic class and the partition should therefore do nothing. It reproduces the baseline saturation rate of 0.70 on the mesh and 0.28 on the ring, which is the expected null result and confirms that the measured hot-node gains come from isolation rather than from an artefact of the implementation.

13. UNIT-TEST DESIGN AND RESULTS

The Week 2 suite still passes unchanged: 35 checks across 7 test functions. The Week 3 suite adds 97 checks across 12 test functions covering the traffic generators, the simulation core, the accounting and the innovation. The project's own central hypothesis is included as an assertion rather than left as a claim in prose, so that a regression which reversed the ring-versus-mesh result would fail the build.

TEST FUNCTION	COVERAGE
test_traffic_generator	Hot set derived from the seed, not hard-coded; no packet addresses its own source; uniform distribution flat to within 8% over 200 000 draws; hot share in the expected range; each hot node receives more than three times a non-hot node.
test_reproducibility	Two runs with the same seed produce identical generated, delivered, latency-sum and drop counts.
test_packet_conservation	The conservation identity holds at four injection rates on both topologies and both traffic patterns.
test_zero_loss_below_saturation	Zero drops and full retirement of every measurement packet at loads below saturation, for four configurations.
test_zero_load_latency	Measured mean hop counts match N/4 and 2(k+1)/3; latency never below serialisation plus one cycle per hop; the mesh is faster unloaded.
test_throughput_tracks_offered	Accepted throughput is within 5% of offered load at four rates below saturation.
test_latency_monotonic	Average latency never decreases as offered load rises.
test_ring_makes_progress_at_overload	The ring still delivers packets at an offered load of 0.80, which fails if the dateline VC rule is removed and the ring deadlocks.
test_mesh_outperforms_ring	At 0.40 under uniform traffic the mesh is stable and the ring is saturated; hot-node traffic raises latency on both topologies relative to uniform at the same load.
test_vc_mode_configuration	Each mode reports the correct VC and queue counts, and total source-queue capacity is identical across modes.
test_class_isolation_removes_hol_blocking	Class isolation cuts background latency to under a fifth of baseline on both topologies; the hot flow's throughput does not regress; total throughput does not regress; conservation still holds.
test_class_isolation_neutral_under_uniform	Under uniform traffic the partition changes latency by less than 15% and no hot class exists.

13.1 Actual test and verification output

```
$ ./test_topology
ALL TESTS PASSED (35 checks across 7 test functions)

$ ./test_simulator
ALL TESTS PASSED (97 checks across 12 test functions)

$ ./run_sweep
-- Week 2 static metrics re-verified --
  ring(16) : diameter=8  bisection=2   (expected 8 / 2)
  mesh(4x4): diameter=6  bisection=4   (expected 6 / 4)
```

```
-- Hot-node (database) set derived from seed 1301 --
  hot nodes = {5, 1}   share of all traffic = 50%

-- ring / uniform / baseline_vc2 --
   offered  accepted    avg_lat    bg_lat   hot_lat  dropped  state
   0.20     0.2000      18.6       18.6     0.0      0        stable
   0.24     0.2393      33.6       33.6     0.0      0        stable
   0.28     0.2675      499.0      499.0    0.0      0        stable
   0.32     0.2724      1610.4     1610.4   0.0      0        SATURATED
   => highest loss-free rate 0.28; peak accepted 0.2724

-- mesh / uniform / baseline_vc2 --
   0.60     0.5992      15.9       15.9     0.0      0        stable
   0.70     0.6932      133.8      133.8    0.0      0        stable
   0.80     0.7241      859.2      859.2    0.0      1051     SATURATED
   => highest loss-free rate 0.70; peak accepted 0.7239

Packet-conservation check: PASSED (0 violation(s) across 216 runs)
```

13.2 Coverage limitations to address before Week 4

- Each sweep point is a single seeded run rather than an average over several seeds, so no confidence interval is reported. Repeating the sweep over three or five seeds before the final report would let the saturation points be quoted with an error bar.
- The saturation rate is read off an eighteen-point grid, so it is known only to the width of the bracket between the last stable rate and the first saturated one. A bisection search over injection rate would tighten it.
- The simulator does not yet check that a packet's flits arrive in order at the destination. Ordering is guaranteed by construction because a virtual channel is held for the whole packet, but the property should be asserted rather than argued.
- Routing functions still assume valid source and destination identifiers. Explicit bounds checking would harden the module before it is exposed to a generated trace file.
- Bisection bandwidth is still returned from the topology-specific closed forms rather than measured by a general minimum-cut algorithm. The limitation is deliberate and is now supplemented by the explicit channel-load computation in §11.

14. BUILD, REPRODUCIBILITY AND GIT EVIDENCE

The Week 3 code was reproduced on Windows using w64devkit GCC. Because the implementation and the tests live in separate repository directories, the repository-root commands below are canonical. A Makefile target runs the whole pipeline in order.

```
gcc -O2 -Wall -Wextra -std=c11 -Istudent_implementation \
    -o test_simulator student_implementation/network_topology.c \
                      student_implementation/traffic.c \
                      student_implementation/network_sim.c \
                      tests/test_simulator.c -lm

gcc -O2 -Wall -Wextra -std=c11 -Istudent_implementation \
    -o run_sweep student_implementation/network_topology.c \
                 student_implementation/traffic.c \
                 student_implementation/network_sim.c \
                 tests/run_sweep.c -lm
```

```
./test_topology.exe && ./test_simulator.exe
./dump_traffic.exe 3000
python3 python/gen_datacenter_traffic.py \
        --verify results/traffic_reference_c.csv
./run_sweep.exe | tee results/week3_sweep_console.txt
python3 python/analyze_network_results.py

# or simply:  make week3
```

14.1 Reproducibility record

ITEM	RECORDED VALUE / LOCATION
Assigned configuration and all model parameters	configs/group1_week3_config.txt
Node count / mesh geometry	16 nodes / 4 × 4
Traffic seed and derived hot set	1301; hot nodes {5, 1}
Random-number generator	xorshift64* seeded through SplitMix64, identical in C and Python
Implementation	student_implementation/network_sim.c/.h, traffic.c/.h, network_topology.c/.h
Tests and drivers	tests/test_topology.c, test_simulator.c, verify_topology.c, run_sweep.c, dump_traffic.c
Analysis	python/gen_datacenter_traffic.py, python/analyze_network_results.py, matlab/bisection_latency_model.m
Compiler flags	-O2 -Wall -Wextra -std=c11
Expected unit-test result	35/35 (Week 2) and 97/97 (Week 3) checks pass
Raw results	results/raw/: week3_sweep_results.csv, week3_saturation_summary.csv, traffic_reference_c.csv
Derived tables	results/processed/: week3_results_table.csv, week3_model_vs_measured.csv, week3_innovation_table.csv
Figures	results/figures/: fig1 to fig7
Generated traffic traces	traces/traffic_uniform_seed1301.csv and traces/traffic_hotnode_seed1301.csv
Console log	results/raw/week3_sweep_console.txt
Repository	github.com/somuahes/CPEN438-Project13-Group1

14.2 Week 3 commit evidence

COMMIT	MESSAGE	EVIDENCE
559c343	Add flit-level simulation core with virtual channels and credit-based flow control	network_sim.c and .h
6fde952	Add seeded uniform-random and hot-node traffic generators	traffic.c/.h; python/gen_datacenter_traffic.py
37b1f20	Add Week 3 unit tests and traffic-generator cross-check	tests/test_simulator.c; tests/dump_traffic.c
653b674	Add injection-rate sweep driver and Week 3 build target	tests/run_sweep.c; Makefile
8c0968e	Add analytical model, analysis pipeline and Week 3 results	bisection_latency_model.m; analyze_network_results.py; results/; traces/

COMMIT	MESSAGE	EVIDENCE
6270be4	Add Week 3 report, design specification and AI-use declaration	report/; ai_use_declaration/ (tagged week3)

Every hash above was read from the repository log after the push; none is asserted without verification. Commit 6270be4 carries the tag week3, which marks the exact state of the repository at the Week 3 submission.

15. WEEK 4 HANDOFF

Week 3 stops at a validated deterministic baseline. The following work is deliberately deferred because it belongs to the extension and defence phase:

- Implement the folded-torus topology from Dally and Towles and quantify its latency and throughput advantage over the plain mesh against the additional wiring, estimated using the paper's area-overhead methodology.
- Implement the research-challenge adaptive routing algorithm, choosing among equally short paths on local congestion, and measure its throughput gain over deterministic dimension-order routing under the hot-node pattern specifically. The deterministic baseline must remain strictly single-path so the comparison stays meaningful.
- Repeat the sweep over several seeds so that saturation points can be quoted with confidence intervals, and refine each saturation rate by bisection rather than reading it off the grid.
- Run the analytical model in MATLAB itself and regenerate the model-versus-measurement figures from it.
- Write the final report with the full topology comparison and the packet-loss accounting, and prepare the ten-slide presentation with a substantive component for every team member.
- Rehearse the defence: compute diameter and bisection bandwidth live for an unseen node count, generate a new team-seeded traffic pattern on demand, and diagnose an introduced routing bug.

16. RISKS AND DEFENCE READINESS

RISK	EFFECT	MITIGATION
A single seed per sweep point could make a saturation rate an artefact of one random stream.	A reported saturation point could move under a different seed.	The seed is fixed and recorded, the pattern is verifiably uniform, and a multi-seed repeat is scheduled for Week 4.
The ejection-bandwidth assumption materially affects the hot-node results.	A reader who assumes single-flit ejection would expect different numbers.	The assumption is stated explicitly in §4.4 with its justification, and the parameter is a named constant that can be changed and re-swept live.
Adding virtual channels for the innovation could be mistaken for the source of the gain.	The innovation's contribution could be overstated.	Two controls separate buffering from isolation, and the total queue capacity is held constant across all modes.

RISK	EFFECT	MITIGATION
Above saturation the reported average latency is a lower bound, because some measurement packets never retire.	Post-saturation latency values could be over-read.	The unretired count is reported at every point, and the saturation criterion is based on throughput divergence rather than a latency threshold.
The ring could deadlock if the dateline rule were removed during later refactoring.	Ring throughput would collapse to zero and the mesh advantage would be exaggerated.	A unit test asserts forward progress at an offered load of 0.80.
Traffic could be generated non-uniformly and defeat the mesh's advantage.	The routing implementation would be wrongly blamed, exactly as the instructor notes warn.	The uniform generator is asserted flat to within 8% over 200 000 draws, and the C and Python generators are verified bit-identical.

17. CONCLUSION

Week 3 turned the structurally verified Week 2 topology module into a working cycle-accurate flit-level simulator and used it to answer the question Project 13 actually asks: how much does topology choice matter, and under what traffic. The simulator integrates the Week 2 routing functions unchanged, models wormhole flow control with virtual channels and credit-based backpressure, and accounts for every packet it generates. Across 216 runs the conservation identity never failed and no packet was lost below any reported saturation point.

Under uniform-random traffic the 4 × 4 mesh sustained 0.70 flits per node per cycle against the 16-node ring's 0.28, a 2.5× advantage that follows directly from having twice the bisection bandwidth and a smaller diameter. Under the hot-node-skewed workload both topologies fell to 0.20, because the constraint moves off the bisection cut and onto the handful of channels feeding the database nodes — a result that costs the mesh most of its advantage and is the more realistic and more revealing of the two cases. An analytical channel-load model written from topology structure alone predicted the hot-node saturation points to within 9 to 16 percent and the uniform ones to within the 25 to 33 percent gap normally attributed to finite buffering, and correctly identified that the classical bisection bound is the wrong bound for skewed traffic.

The innovation component demonstrated that the damage skewed traffic does to unrelated flows is largely an artefact of shared buffering rather than a physical necessity. Separating the two traffic classes at both the injection queue and the router input buffers reduced background-traffic latency by 71× on the mesh and 60× on the ring at the hot-node knee, and raised peak accepted throughput on the mesh by 48 percent, while two controls confirmed that neither extra buffering nor the queue split alone accounts for the effect. Week 4 will extend the study to the folded torus and to adaptive routing, and will strengthen the statistical basis of the saturation measurements.

18. REFERENCES

[1] W. J. Dally and B. Towles, "Route packets, not wires: On-chip interconnection networks," in Proc. 38th Design Automation Conf. (DAC), Las Vegas, NV, USA, 2001, pp. 684–689, doi: 10.1109/DAC.2001.935594.

[2] W. J. Dally and B. Towles, Principles and Practices of Interconnection Networks. San Francisco, CA, USA: Morgan Kaufmann, 2003. The dateline virtual-channel rule used for the ring in §4.3 and the channel-load saturation bound used in §11 both follow this text.

[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017. Appendix F covers interconnection networks, including the diameter and bisection-bandwidth definitions used throughout.

[4] H. Esmaeilzadeh, E. Blem, R. St. Amant, K. Sankaralingam, and D. Burger, “Dark silicon and the end of multicore scaling,” in Proc. 38th Annu. Int. Symp. Computer Architecture (ISCA), 2011, pp. 365–376, doi: 10.1145/2000064.2000108.

[5] W. J. Dally, “Performance analysis of k-ary n-cube interconnection networks,” IEEE Trans. Comput., vol. 39, no. 6, pp. 775–785, Jun. 1990, doi: 10.1109/12.53599. The basis for relating topology, routing and measured saturation behaviour.

[6] W. J. Dally, “Virtual-channel flow control,” IEEE Trans. Parallel Distrib. Syst., vol. 3, no. 2, pp. 194–205, Mar. 1992, doi: 10.1109/71.127260. The foundation of the virtual-channel scheme evaluated in §12.

[7] G. Marsaglia, “Xorshift RNGs,” J. Statistical Software, vol. 8, no. 14, pp. 1–6, 2003, doi: 10.18637/jss.v008.i14. The generator used for reproducible traffic in §5.